数据库管理系统原理与实现

杜小勇 陈红 卢卫 主编

深圳技术大学
大数据与互联网学院

第4章 高级SQL

- 4.1 复杂SQL查询
- 4.2 数据库完整性
- 4.3 数据库安全性
- 4.4 数据库编程
- 4.5 小结

4.1 复杂SQL查询

- 4.1.1 嵌套查询
- 4.1.2 递归查询
- 4.1.3 基于派生表的查询

4.1.1 嵌套查询

本章嵌套查询只讨论在WHERE子句中出现的查询语句。WHERE子句可以包含比较类谓词（predicate），包括comparison、between、in、like、null、quantified等多种形式。

<predicate> ::=
<comparison predicate>比较
<between predicate>范围
<in predicate>包含
<like predicate>模糊匹配
<null predicate>空值 **IS NULL, IS NOT NULL**
<quantified comparison predicate>存在
<exists predicate>存在
<unique predicate> 列属性unique
<match predicate> REGEXP
<overlaps predicate> 判断区间数据如日期等，mysql无overlaps

嵌套查询（续）

- 嵌套查询概述
  - 一个SELECT-FROM-WHERE语句称为一个查询块
  - 将一个查询块嵌套在另一个查询块的WHERE子句或HAVING短语的条件中的查询称为嵌套查询
  - SQL语言允许多层嵌套查询
    - 即一个子查询中还可以嵌套其他子查询
  - 子查询的限制
    - 不能使用ORDER BY子句

4.1.1 嵌套查询

[例4.1]查询哪些供应商供应了数码产品类别的商品。

```
SELECT *                                    /*外层查询/父查询*/
    FROM Suppliers
    WHERE SID IN
    (
        /*内层查询/子查询*/
        SELECT SID
        FROM Products
        WHERE Category LIKE '数码产品'
);
```

子查询由包含谓词IN引出。

在此查询中，子查询不依赖于父查询，这样的子查询称为不相关子查询。子查询的结果用于建立父查询的查找条件

反之，则称为相关子查询，相关子查询将结合EXISTS子查询介绍。

查询结果：

SID	SName	City	PID
S001	华北电子厂	北京	P0001

4.1.1 嵌套查询

上述查询也可以用连接来实现。

```
SELECT Suppliers.*
        FROM Suppliers
        JOIN Products ON Suppliers.SID= Products.SID
        WHERE Products.Category LIKE ‘数码产品’ ;
```

4.1.1 嵌套查询

1.带有IN谓词的子查询

2.带有比较运算符的子查询

3.带有ANY（SOME）或ALL谓词的子查询

4.带有EXISTS谓词的子查询

带有IN谓词的子查询（续）

[例,第6版]查询选修了课程名为“信息系统”的学生学号和姓名

```
SELECT Sno,Sname
FROM    Student
WHERE Sno  IN
        (SELECT Sno
         FROM    SC
         WHERE  Cno IN
              (SELECT Cno
                FROM Course
                WHERE Cname= '信息系统'
              )
        );
```

③ 最后在Student关系中取出Sno和Sname

② 然后在SC关系中找出选修了3号课程的学生学号

① 首先在Course关系中找出“信息系统”的课程号，为3号

带有IN谓词的子查询（续）

用连接查询实现：

```
SELECT Sno,Sname
FROM    Student,SC,Course
WHERE Course.Cname='信息系统 '   AND
            Course.Cno = SC.Cno AND
            SC.Sno = Student.Sno;
```

/* 以上顺序一般不改变性能，调优一般使用join*/

大部分情况性能高于嵌套查询，需要具体分析
但连接查询不能代替EXISTS嵌套查询

4.1.1 嵌套查询

1.带有IN谓词的子查询

2.带有比较运算符的子查询

3.带有ANY（SOME）或ALL谓词的子查询

4.带有EXISTS谓词的子查询

带有比较运算符的子查询（续）

[例 ,第6版]找出每个学生超过他选修课程平均成绩的课程号。

相关子查询：隐含两层循环

```
SELECT Sno, Cno
FROM    SC  x
WHERE Grade >=(SELECT AVG（Grade)
                FROM  SC y
                  WHERE y.Sno=x.Sno);
```

Q：子查询是否需要加group by x.sno或y.sno?

A：没必要。

带有比较运算符的子查询（续）

- 可能的执行过程
  - 从外层查询中取出SC的一个元组x，将元组x的Sno值（201215121）传送给内层查询。

```
SELECT AVG(Grade)
FROM SC y
WHERE y.Sno='201215121';
```

带有比较运算符的子查询（续）

- 可能的执行过程（续）
  - 执行内层查询，得到值88（近似值），用该值代替内层查询，得到外层查询：

```
SELECT Sno,Cno
FROM   SC x
WHERE  Grade >=88;
```

带有比较运算符的子查询（续）

❖可能的执行过程（续）

- 执行这个查询，得到

(201215121,1)

(201215121,3)

然后外层查询取出下一个元组重复做上述①至③步骤，直到外层的SC元组全部处理完毕。结果为:

(201215121,1)

(201215121,3)

(201215122,2)

带有比较运算符的子查询（续）

[课堂练习]找出每门课最高分的学生学号。

```
SELECT Cno,Sno,
FROM    SC  x
WHERE Grade =(SELECT MAX(Grade)
                FROM  SC y
                  WHERE y.Cno=x.Cno);
```

相关子查询：隐含两层循环

4.1.1 嵌套查询

1.带有IN谓词的子查询

2.带有比较运算符的子查询

3.带有ANY（SOME）或ALL谓词的子查询

4.带有EXISTS谓词的子查询

带有ANY（SOME）或ALL谓词的子查询（续）

表3.7 ANY（或SOME），ALL谓词符合自然语言描述习惯，与聚集函数、IN谓词的等价转换关系

	=	<>或!=	<	<=	>	>=
ANY	IN	--	<MAX	<=MAX	>MIN	>= MIN
ALL	--	NOT IN	<MIN	<= MIN	>MAX	>= MAX

带有ANY（SOME）或ALL谓词的子查询 （续）

使用ANY或ALL谓词时必须同时使用比较运算

语义为:

> ANY	大于子查询结果中的某个值(结果集尽可能大)
> ALL	大于子查询结果中的所有值
< ANY	小于子查询结果中的某个值(结果集尽可能大)
< ALL	小于子查询结果中的所有值
>= ANY	大于等于子查询结果中的某个值
>= ALL	大于等于子查询结果中的所有值

带有ANY（SOME）或ALL谓词的子查询（续）

使用ANY或ALL谓词时必须同时使用比较运算

语义为（续）

<= ANY 小于等于子查询结果中的某个值

<= ALL 小于等于子查询结果中的所有值

= ANY 等于子查询结果中的某个值（任意）

=ALL 等于子查询结果中的所有值（通常没有实际意义）

!=（或<>）ANY 不等于子查询结果中的某个值

!=（或<>）ALL 不等于子查询结果中的任何一个值

带有ANY（SOME）或ALL谓词的子查询 （续）

[例 ,第6版] 查询非计算机科学系中比计算机科学系任意一个学生年龄小的学生姓名和年龄

```
SELECT Sname,Sage
FROM    Student
WHERE Sage < ANY (SELECT  Sage
                  FROM    Student
                  WHERE Sdept= ' CS ')
 AND Sdept <> ‘CS ' ;          /*父查询块中的条件 */
```

带有ANY（SOME）或ALL谓词的子查询（续）

结果：

Sname	Sage
王敏	18
张立	19

执行过程：

（1）首先处理子查询，找出CS系中所有学生的年龄，构成一个集合（20,19）

（2）处理父查询，找所有不是CS系且年龄小于20 或 19的学生

带有ANY（SOME）或ALL谓词的子查询 （续）

用聚集函数实现

```
SELECT Sname,Sage
FROM   Student
WHERE Sage <
                    (SELECT MAX（Sage）
                      FROM Student
                      WHERE Sdept= 'CS ')
  AND Sdept <> ' CS ';
```

带有ANY（SOME）或ALL谓词的子查询 （续）

[例 ,第6版] 查询非计算机科学系中比计算机科学系所有学生年龄都小的学生姓名及年龄。

方法一： 用ALL谓词

```
SELECT Sname,Sage
 FROM Student
 WHERE Sage < ALL
                (SELECT Sage
                 FROM Student
                 WHERE Sdept= ' CS ')
  AND Sdept <> ' CS ';
```

带有ANY（SOME）或ALL谓词的子查询 （续）

方法二：用聚集函数

```
SELECT Sname,Sage
FROM Student
WHERE Sage <
                    (SELECT MIN(Sage)
                     FROM Student
                     WHERE Sdept= ' CS ')
  AND Sdept <>' CS ';
```

4.1.1 嵌套查询

1.带有IN谓词的子查询

2.带有比较运算符的子查询

3.带有ANY（SOME）或ALL谓词的子查询

4.带有EXISTS谓词的子查询

带有EXISTS谓词的子查询

- EXISTS谓词
  - 存在量词 ∃
  - 带有EXISTS谓词的子查询不返回任何数据，只产生逻辑真值“true”或逻辑假值“false”。
    - 若内层查询结果非空，则外层的WHERE子句返回真值
    - 若内层查询结果为空，则外层的WHERE子句返回假值
  - 由EXISTS引出的子查询，其目标列表达式通常都用 *，因为带EXISTS的子查询只返回真值或假值，给出列名无实际意义。

带有EXISTS谓词的子查询（续）

- **NOT EXISTS谓词**
  - 若内层查询结果非空(true)，则外层的WHERE子句返回假值
  - 若内层查询结果为空(False)，则外层的WHERE子句返回真值

带有EXISTS谓词的子查询（续）

[例4.2]查询哪些订单是由来自北京的顾客创建的。

连接查询方式:

```
SELECT Orders.*
FROM Orders, Customers
WHERE Orders.CID = Customers.CID  AND
Customers.City = '北京';
```

4.1.1 嵌套查询

[例4.2]查询哪些订单是由来自北京的顾客创建的。(本例没必要用相关子查询，工作中可能为性能调优)

```
SELECT *
        FROM Orders O
        WHERE EXISTS (
                SELECT *
                FROM Customers C
                WHERE C.City LIKE '北京'
                AND C.CID = O.CID
        );
```

子查询依赖于父查询，是相关子查询，如关系Orders同时出现在外层和内层。

由EXISTS引出的子查询，构成一个逻辑表达式，输出True或False。若子查询结果不为空，EXISTS子查询返回True；否则，返回False。

查询结果：

订单ID (OID)	顾客ID (CID)	创建时间 (CreateTime)
O001	1001	2023-1-1 18:00:10

4.1.1 嵌套查询

相关子查询执行时，须遍历外层查询关系的所有元组。

- 将每个元组的值代入到内层查询中。然后执行内层查询，若内层查询结果不为空/true(exists)，则该元组满足查询条件，否则就不满足。
- 然后再取外层表的下一个元组，重复这一过程，直至外层表全部检查完为止。

4.1.1 嵌套查询

取出Orders第一个元组:
t1=(O001,1001，2023-1-1 18:00:10)

↓

将t1代入子查询，则子查询变为：
SELECT * FROM Customers WHERE C.CID = ‘1001’

↓

执行内层子查询，结果不为空，EXISTS子句返回True

↓

t1满足查询要求，得到第一个结果

带有EXISTS谓词的子查询（续）

[例 ,第6版] 查询没有选修1号课程的学生姓名。

```
SELECT Sname
FROM     Student
WHERE NOT EXISTS
            (SELECT * #不存在(没选)才是T，存在(选)为F
             FROM SC
             WHERE Sno = Student.Sno AND Cno='1');
```

Q：改成这样是否可以?

```
SELECT Sname  FROM     Student,SC
    WHERE  SC.Sno = Student.Sno AND Cno!='1';
```

A：错误，逐行记录筛选！如果选修了多门课，含1号课程及其他课程，其他课程会导致该生符合以上条件。

带有EXISTS谓词的子查询（续）

改写：[例,第6版] 查询没有选修1号课程的学生姓名。

```
SELECT   Sname
    FROM     Student
    WHERE Sno NOT IN
        (SELECT distinct Sno  #选了1号课
            FROM SC
            WHERE Cno='1');
```

带有EXISTS谓词的子查询（续）

- 不同形式的查询间的替换
  - 一些(比如元组涉及空值)带EXISTS或NOT EXISTS谓词的子查询不能被其他形式的子查询等价替换
  - 所有带IN谓词、比较运算符、ANY和ALL谓词的子查询都能用带EXISTS谓词的子查询等价替换
  - （需要具体分析性能，有的自动优化，EXISTS经常性能占优，一般更适合于外表小而内表大的情况；而IN适用于不相关子查询，或者一般适于外查询表大而内查询表小的情况；）

https://docs.oracle.com/cd/E11882_01/server.112/e41084/conditions013.htm#SQLRF52169

实际与DB版本(底层的hash join、buffer等技术)、索引、数据集均有关，还需要根据实际情况测试对比。

带有EXISTS谓词的子查询（续）

[例 ,第6版]查询与“刘晨”在同一个系学习的学生。

可以用带EXISTS谓词的子查询替换IN:

```
SELECT Sno,Sname,Sdept
  FROM Student S1
  WHERE EXISTS
            (SELECT *
              FROM Student S2
              WHERE S2.Sdept = S1.Sdept AND
                    S2.Sname = '刘晨');
                          #相关子查询
```

```
WHERE Sdept  IN
      (SELECT Sdept
FROM Student
WHERE Sname= ' 刘晨 ');
#不相关子查询。=any(…)
```

带有EXISTS谓词的子查询（续）

- 用EXISTS/NOT EXISTS实现全称量词（难点）
  - SQL语言中没有全称量词∀ （For all）
  - 可以把带有全称量词的谓词转换为等价的带有存在量词的谓词：

$$(\forall x)\ P \equiv \neg\ (\exists x\ (\neg P))$$

集合证明(双重否定)：

带有EXISTS谓词的子查询（续）

[例1] 查询选修了全部课程的学生姓名。

查询这样的学生，没有一门课程是他不选修的。

NOT EXISTS 1(不存在)他没选修的课。

NOT EXISTS 2(不存在)该生选了该课的元组。

Student 表

Sno	Sname	Ssex	Sdept
200215121	李勇	男	CS
200215122	刘晨	女	IS
200215123	王敏	女	IS

SC 表

Sno	Cno	Grade
200115121	1	90
200215121	2	85
200215121	3	88
200215122	2	90
200215122	3	80

子查询1	空
WHERE	true
子查询2	
WHERE 1	

Course 表

Cno	Cname	Cpno	Ccredit
1	数据库	3	4
2	数学		2
3	C语言	1	4

结果

Sname
李勇

中国人民大学 数据库系统概论

An Introduction to Database Systems

3_EXISTS_(new).mp4

- 爱课程网数据库系统概论数据查询节动画《EXISTS子查询》

带有EXISTS谓词的子查询（续）

[例 3.65]查询至少选修了学生20180002选修的全部课程的学生号码(考试必考)。

解题思路:

- 用逻辑蕴涵表达：查询学号为x的学生，对所有的课程y，只要20180002学生选修了课程y，则x也选修了y。
- 形式化表示:

  用P表示谓词 “学生20180002选修了课程y”

  用q表示谓词 “学生x选修了课程y”

  则上述查询为: $(\forall y)\ p \rightarrow q$

带有EXISTS谓词的子查询（续）

（∀x）P ≡ ¬ （∃ x（¬ P））(双重否定)

■等价变换：

（∀y） p → q ≡ ¬ （∃y （¬ （p → q ））（双重否定)

≡ ¬ （∃y （¬ （¬ p ∨ q） ））（真值表）

≡ ¬ ∃y （p ∧ ¬q） （反演律/摩根定理）

■变换后语义：不存在这样的课程y，学生20180002选修了y，而学生x没有选y。

带有EXISTS谓词的子查询（续）

■用NOT EXISTS谓词表示：

```
SELECT DISTINCT Sno
FROM SC SCX
WHERE NOT EXISTS
        (SELECT *
         FROM SC SCY
         WHERE SCY.Sno = ' 20180002 '  AND
               NOT EXISTS
               (SELECT *          #遍历完SC表，全选修则为F
                FROM SC SCZ
                WHERE SCZ.Sno=SCX.Sno AND
                      SCZ.Cno=SCY.Cno)); #P谓词
```

（∀y）p → q ≡ ¬ ∃y（p∧¬q）

用P表示谓词 “学生20180002选修了课程y”

用q表示谓词 “学生x选修了课程y”

则上述查询为:（∀y） p → q

不存在这样的课程y，学生**20180002**选修了y，而学生x没有选y

Q：增加条件：20180002学生成绩大于70分的课程，并且只求出CS系的学生

4.1 复杂SQL查询

- 4.1.1 嵌套查询
- 4.1.2 递归查询
- 4.1.3 基于派生表的查询

（附录）WITH…AS 临时结果表

使用WITH..AS创建临时结果集（Common Table Expressions, CTE）），可用于简化查询逻辑，提高代码可阅读性。例如：

```
WITH
  cte1 AS (SELECT a, b FROM table1),
  cte2 AS (SELECT c, d FROM table2)

SELECT b, d FROM cte1 JOIN cte2
WHERE cte1.a = cte2.c;
```

在查询执行之前，先按照WITH..AS的定义构建cte1和cte2两个临时结果集。

执行查询时，像使用基本表和视图一般直接使用临时结果集即可。

4.1.2 递归查询

递归查询(公用表表达式（Common Table Expressions, CTE））可用于实现循环处理具有层级关系的数据，语法格式如下：

```
WITH RECURSIVE temporary_table AS (
        SEED QUERY
        UNION [ALL]
        RECURSIVE QUERY
)
```

查询执行时，先执行SEED Query，其结果作为临时表temporary_table内容，接下来执行RECURSIVE QUERY，注意temporary_table必须要出现在RECURSIVE QUERY中。如果RECURSIVE QUERY查询结果不为空，根据其查询结果更新临时表temporary_table内容，并重复执行RECURSIVE QUERY。

4.1.2 递归查询

[例4.3]输出1到100之和。

```
WITH RECURSIVE t(n) AS (
        VALUES (1) #低版本可用SELECT 1
        UNION
        SELECT n+1 FROM t WHERE n < 100
)
SELECT sum(n) FROM t; #对生成的t表求和
```

查询结果：

sum
5050

临时表为t，只有一个属性列n。

SEED QUERY是VALUES(1)，这是特殊查询，返回只包含数字1的一行(需MySQL版本高于8.0.19)。执行完SEED QUERY之后，临时表内容只有一行，即数字1，初始值。

执行一次RECURSIVE QUERY后，得到新的一行，即数字2。根据查询结果更新临时表内容，临时表中将保留每次RECURSIVE QUERY执行的结果(新建一行)。重复执行RECURSIVE QUERY，直到结果为空。按条件完成递归。

4.1.2 递归查询

[例4.4，难点]查找属于数码产品的所有类别。假设在我们的网购数据库中，类别有子类，即有一个新的关系 Categories(Category, Parent)。例如存在三行记录<智能手表，数码产品>，<智能手机，手机>，<手机，数码产品>。表示智能手机是手机的子类，而手机是数码产品的子类。

```
WITH RECURSIVE RS(Category, Parent) AS (
        SELECT * FROM Categories WHERE Parent = '数码产品'
        UNION
        SELECT Categories.* FROM RS JOIN Categories ON RS.Category =
Categories.Parent);
```

先执行SEED QUERY，得到两行记录，即<智能手表，数码产品>和<手机，数码产品>，存入临时表RS。

Q：为何不用自身连接方式？

执行RECURSIVE QUERY，找到间接属于数码产品(Parent也是Category)，得到一行记录<智能手机，手机>。

查询结果：

Category	Parent
智能手表	数码产品
智能手机	手机
手机	数码产品

4.1.2 递归查询

Q：为何不用自身连接方式？

A： 假设存在以下记录，则有4个层级(从右到左看)： <折叠5G智能手机， 5G智能手机>， <5G智能手机，智能手机>， <智能手机，手机>，<手机，数码产品>

或者间接先修课(比如间接和间接的间接一起查询)

COALESCE()函数用于返回指定系列表达式中的第一个非null值

自身表连接方式的代码容易有局限性，扩展性差，而且可读性差，一般仅用于两个层级(CTE缺省为1千层级，但易有性能问题)

```
SELECT DISTINCT
    COALESCE(c4.Category, c3.Category, c2.Category, c1.Category) AS Category,
    COALESCE(c4.Parent, c3.Parent, c2.Parent, c1.Parent) AS Parent
FROM
    Categories c1
-- 第一次连接，查找数码产品的直接子类
LEFT JOIN Categories c2 ON c1.Category = c2.Parent
-- 第二次连接，查找数码产品直接子类的子类
LEFT JOIN Categories c3 ON c2.Category = c3.Parent
-- 第三次连接，查找更深一层的子类
LEFT JOIN Categories c4 ON c3.Category = c4.Parent
WHERE
    c1.Parent = '数码产品'
    OR c2.Parent = '数码产品'
    OR c3.Parent = '数码产品'
    OR c4.Parent = '数码产品';
```

4.1.2 递归查询

假设存在以下记录，则有4个层级(从右到左看)： <折叠5G智能手机， 5G智能手机>， <5G智能手机，智能手机>， <智能手机，手机>，<手机，数码产品>

CTE常见筛选查询场景，再结合各种统计分析:

- 查询子节点 含自己：即折叠5G智能手机、5G智能手机、智能手机、手机、数码产品
- 查询子节点 不含自己：以上去除数码产品
- 查询某类型的子+父节点 含自己：即示例数据
- 查询某类型的子+父节点 不含自己：以上，去除<手机，数码产品>

。 。 。 。

4.1 复杂SQL查询

- 4.1.1 嵌套查询
- 4.1.2 递归查询
- 4.1.3 基于派生表的查询

4.1.3 基于派生表的查询

子查询不仅可以出现在 WHERE子句中，还可以出现在 FROM 子句中，这时子查询临时生成的派生表（ derived table）成为主查询的查询对象。

[例4.5]查询价格低于**所有商品平均价格**的商品。

查询语句生成了一个派生表，名称为AVG_Price。但仅存了一个值，因此没必要用派生表方式。

```
SELECT *
        FROM Products CROSS JOIN (SELECT AVG(Price) FROM Products) AS
AVG_Price (AP)
        WHERE Products.Price < AVG_Price.AP);
```

查询结果：

商品ID (PID)	商品名称 (PName)	价格 (Price)	类别 (Category)	供应商ID (SID)
P0004	数据库教材	48	书籍	S002
P0005	流浪太阳	65	书籍	S002

4.1.3 基于派生表的查询

[例4.5]查询价格低于**所有商品平均价格**的商品。

改写:

```
SELECT *
FROM Products
WHERE Price < (SELECT AVG(Price) FROM
Products); #一个值，可以不放在FROM子句。
```

4.1.3 基于派生表的查询

[例,第6版]找出每个学生超过他自己选修课程平均成绩的课程号

不使用相关子查询

```
SELECT Sno, Cno
FROM SC, (SELECT Sno, Avg(Grade)
          FROM SC
          GROUP BY Sno)
          AS  Avg_sc(Avg_sno,Avg_grade)/*教材的派生表列名方式不适合mysql 5.7，列的别名需要在select后*/
WHERE SC.Sno = Avg_sc. Avg_sno
  and SC.Grade >=Avg_sc.Avg_grade
```

基于派生表的查询（续）

- 如果子查询中没有聚集函数，派生表可以不指定属性列，子查询SELECT子句后面的列名为其缺省属性。

[例,第6版]查询所有选修了1号课程的学生姓名，可以用如下查询完成，实际还是多表连接：

```
SELECT Sname
FROM    Student,
        (SELECT Sno FROM SC WHERE Cno=' 1 ') AS SC1
WHERE  Student.Sno=SC1.Sno;
```

SQL扩展之常见互联网业务场景

教材无，有OJ实验，期末考试则作为附加题出现(A+的关键)。

MySQL函数

https://dev.mysql.com/doc/refman/8.0/en/functions.html
https://www.runoob.com/mysql/mysql-functions.html

MySQL函数是MySQL数据库提供的内置函数。这些内置函数可以帮助用户更加方便地处理表中的数据。

MySQL内置函数类别及作用

函 数	作用
数学函数	用于处理数字。这类函数包括绝对值函数、正弦函数、余弦函数和获取随机数函数等
字符串函数	用于处理字符串。其中包括字符串连接函数、字符串比较函数、字符串中字母大小写转换函数等
日期和时间函数	用于处理日期和时间。其中包括获取当前时间的函数、获取当前日期的函数、返回年份的函数和返回日期的函数等
条件判断函数	用于在SQL语句中控制条件选择。其中包括IF语句、CASE语句和WHEN语句等
系统信息函数	用于获取MySQL数据库的系统信息。其中包括获取数据库名的函数、获取当前用户的函数和获取数据库版本的函数等
加密函数	用于对字符串进行加密解密。其中包括字符串加密函数和字符串解密函数等
其他函数	包括格式化函数和锁函数等

substr()函数

1、substr(str,pos);

```
SELECT SUBSTR(‘2018-08-17’,6); #类似英文，下标从1开始，而不是0
```

SUBSTR('2018-08-17',6)
08-17

2、substr(str from pos);

```
SELECT SUBSTR('2018-08-17' FROM 6);
```

SUBSTR('2018-08-17' FROM 6)
08-17

3、substr(str,pos,len);//str:字符串，pos：起始位置，len：截断长度

```
SELECT SUBSTR('2018-08-17',6,7);
```

SUBSTR('2018-08-17',6,7)
08-17

4、substr(str from pos for len);

互联网场景例题：例题：查6月份的月活数据MAU

用户登录记录表，表结构如下，用户每成功登录一次，就会记录一次

td_load_rcd(用户登录记录表)		
usr_id(用户id)	load_dt(登录日期)	load_tm(登录时间)
C10000	2020/6/9	16:16:13

```
Select substr(load_dt,1,7) load_month,count(distinct
usr_id) mau     /*查6月份的月活数据，活跃用户数是去重用户数*/
from td_load_rcd
where substr(load_dt,6,2)=6 /*只选6，并未要求按某年查，如果每年分表则加上年的筛选*/
group by substr(load_dt,1,7);# 按各月分组
```

load_month	mau
2020-06	19370

思考题：sql查2020年6月的月活数据MAU和同比增长率

CONCAT(s1,s2...sn)	字符串 s1,s2 等多个字符串合并为一个字符串
ROUND(x,y)	按Y位小数，对X四舍五入
DATE()	从日期或日期时间表达式中提取日期值
CAST(x AS type)	转换数据类型CAST(a.下单时间 AS DATE);

例题：查2020年6月的月活数据MAU和同比增长率

```
with b as (
  Select count(distinct usr_id) premau /*一个值*/
  from td_load_rcd
  where substr(load_dt,1,7)=201906 /*上一年6月*/
  group by substr(load_dt,1,7);  /*按月分组*/
)
Select substr(load_dt,1,7) load_month,
   count(distinct usr_id) mau,
   concat(cast((count(distinct usr_id)- b.premau)/b.premau*100) as char),'%')
from td_load_rcd
where substr(load_dt,1,7)=202006
group by substr(load_dt,1,7);  /*按月分组*/
```

MySQL条件判断函数

条件函数用来在SQL语句中进行条件判断。根据不同的条件，执行不同的SQL语句。MySQL支持的条件判断函数及作用

函 数	作用
IF(expr,v1,v2)	如果表达式expr成立，则执行v1；否则执行v2
IFNULL(v1,v2)	如果v1不为空，则显示v1的值；否则显示v2的值
CASE WHEN expr1 THEN v1 [WHEN expr2 THEN v2 …][ELSE vn] END	case表示函数开始，end表示函数结束。如果表达式expr1成立，则返回v1的值(不能是执行语句)；如果表达式expr2成立，则返回v2的值。依次类推，最后遇到else时，返回vn的值。它的功能与PHP中的switch语句类似
CASE expr WHEN e1 THEN v1 [WHEN e2 THEN v2 …][ELSE vn] END	case表示函数开始，end表示函数结束。如果表达式expr取值为e1，则返回v1的值；如果表达式expr取值为e2，则返回v2的值，依次类推，最后遇到else，则返回vn的值

7月上旬的工作日期间，各时间段的月活分布，通勤（7：00-9：00、 18：00-20： 00），午休（12： 00-14：00） ，临睡（22：00-1：00），哪段时间的活跃用户数最高？

```
select
case when hour(load_tm) between 7 and 9 or hour(load_tm) between 18 and 20 then 'commute'
when hour(load_tm) between 11 and 12 then 'lunch'
when hour(load_tm) in (22, 23, 0) then 'before_sleep'
end as time_prd  --与group by一样
,count(distinct usr_id) as cst_dt
from
td_load_rcd where load_dt in ('2020-07-01', '2020-07-02', ‘2020-07-03’, ‘2020-07-06’, ‘2020-07-07’,‘2020-07-08’, ‘2020-07-09’, ‘2020-07-10’) ) /*工作日，集合，可以用函数实现*/
group by
case when hour(load_tm) between 7 and 9 or hour(load_tm) between 18 and 20 then 'commute'
when hour(load_tm) between 11 and 12 then 'lunch'
when hour(load_tm) in (22, 23, 0) then 'before_sleep'
end -- 不需要as 。。。
```

time_prd	cst_dt
before_sleep	4783
commute	4941
lunch	2908

2级推送广告对象数量：单日登录次数大于等于5次的用户数量

td_load_rcd(用户登录记录表)		
usr_id(用户id)	load_dt(登录日期)	load_tm(登录时间)
C10000	2020/6/9	16:16:13

```
select count(distinct usr_id) as adusr_cnt
from
(
   select usr_id, load_dt, count(*) as
   load_times /*每个用户每天登录次数*/
   from td_load_rcd
   group by usr_id, load_dt
   having count(*) >=5

)as tmp;
```

adusr_cnt
95

MySQL从8.0开始支持窗口函数，这个功能在大多商业数据库和部分开源数据库中早已支持，有的也叫分析函数。

什么叫窗口？

窗口的概念非常重要，它可以理解为记录集合，窗口函数也就是在满足某种条件的记录集合上执行的特殊函数。对于每条记录都要在此窗口内执行函数。

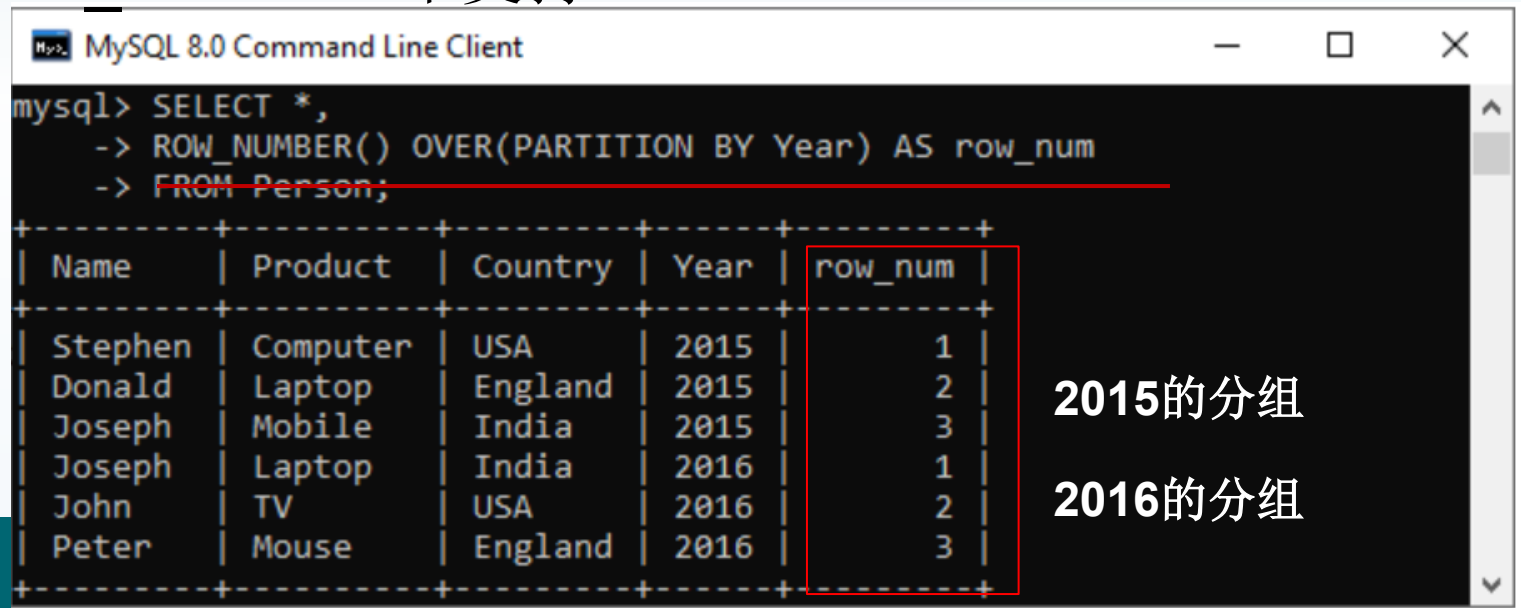
按照功能划分，可以把MySQL支持的窗口函数分为如下几类：

- 序号函数： row_number() / rank() / dense_rank()
- 分布函数： percent_rank() / cume_dist()
- 前后函数： lag() / lead()
- 头尾函数： first_val() / last_val()
- 其他函数： nth_value() / nfile()

窗口函数的基本用法：函数名（[expr]） over子句

over是关键字，用来指定函数执行的窗口范围，如果后面括号中什么都不写，则意味着窗口包含满足where条件的所有行，窗口函数基于所有行进行计算；如果不为空，则支持以下四种语法来设置窗口：

1. window_name：给窗口指定一个别名
2. partition子句：窗口按照那些字段进行分组，窗口函数在不同的分组上分别执行。**下例按照Year进行了分组，生成从1开始的顺序编号**
3. order by子句：窗口函数将按照排序后的记录顺序进行编号
4. frame子句：滑动窗口的范围指定，有两种方式，基于行和基于范围，静态窗口函数比如ROW_NUMBER不支持



row_number()： 不会有重复的排序数值

数值	排序序号
1	1
1	2
2	3

rank()： 考虑了相等值后排序名次，即常说的名次，不一定有第2名

数值	排序序号
1	1
1	1
2	3

dense_rank()： 序号只按值排序，不考虑值相等的数量(值排名，比如可确保有第2名)

数值	排序序号
1	1
1	1
2	2

互联网场景业务常用的日期、时间函数

DATE()	从日期或日期时间表达式中提取日期值
DATE_SUB(start_date,INTERVAL expr unit);	从DATE或DATETIME值中减去时间值(或间隔)，expr 可以是某列，也可以说表达式/常数；DATE_ADD()则为加；
DATEDIFF(date1,date2)	返回两个日期之间的天数，只有值的日期部分参与计算，date1 和 date2 参数是合法的日期或日期/时间表达式
TIMEDIFF(time1,time2)	返回两个时间 之间的 时间差。time1和 time2参数是合法的时间或表达式
TIMESTAMPDIFF(unit,time1,time2)	返回两个时间 之间的 单位时间差
CURDATE()/CURTIME()	返回当前的日期/时间

unit 参数可以是下列值：MICROSECOND毫秒 SECOND秒 MINUTE 分钟 HOUR小时 DAY天 WEEK 周 MONTH月 QUARTER季度 YEAR年 SECOND_MICROSECOND秒-毫秒 MINUTE_MICROSECOND分钟-毫秒 MINUTE_SECOND分钟-秒 HOUR_MICROSECOND小时-毫秒 HOUR_SECOND 小时-秒 HOUR_MINUTE小时-分钟 DAY_MICROSECOND天-毫秒 DAY_SECOND 天-秒 DAY_MINUTE天-分 DAY_HOUR 天-小时 YEAR_MONTH年-月

推送广告对象或者福利：连续登陆7天以上的用户

多种编程方法，但常用**方法是加一个字段，连续登录天数**

一个简化但清晰的示例:

1、因为每天用户登录次数可能不止一次，所以需要先将用户每天的登录日期去重。

2、再用row_number() over(partition by)窗口函数将用户id分组，按照登陆时间进行排序。

3、计算登录日期减去第二步骤得到的结果值，用户连续登陆情况下，每次相减的结果都相同。

4、按照id和日期分组并求和，筛选大于等于7的即为连续7天登陆的用户。

笔试和工作要求：注释写思路。

id	date
1	2019-01-01 00:00:00
1	2019-01-02 00:00:00
1	2019-01-03 00:00:00
1	2019-01-04 00:00:00
1	2019-01-05 00:00:00
1	2019-01-06 00:00:00
1	2019-01-07 00:00:00
1	2019-01-08 00:00:00
4	2019-01-01 00:00:00
2	2019-01-02 00:00:00
3	2019-01-03 00:00:00
4	2019-01-04 00:00:00
5	2019-01-05 00:00:00
6	2019-01-06 00:00:00
7	2019-01-07 00:00:00
8	2019-01-08 00:00:00
9	2019-01-01 00:00:00
9	2019-01-02 00:00:00
9	2019-01-03 00:00:00
9	2019-01-04 00:00:00
9	2019-01-06 00:00:00
9	2019-01-09 00:00:00
9	2019-01-10 00:00:00
9	2019-01-13 00:00:00
1	2019-01-02 15:55:00
2	2019-01-02 15:22:00

1用户登录日期去重，id+日期

```
With A as(select DISTINCT date(date) as 日期,id from login),
```

日期	id
2019-01-01	1
2019-01-02	1
2019-01-03	1
2019-01-04	1
2019-01-05	1
2019-01-06	1
2019-01-07	1
2019-01-08	1
2019-01-01	4
2019-01-02	2
2019-01-03	3
2019-01-04	4
2019-01-05	5
2019-01-06	6
2019-01-07	7
2019-01-08	8
2019-01-01	9
2019-01-02	9
2019-01-03	9
2019-01-04	9
2019-01-06	9
2019-01-09	9

2按照登陆时间进行排序计数

```
B as( select *,row_number() over(PARTITION by id order by 日期) as cum from(A)),
```

日期	id	cum
2019-01-01	1	1
2019-01-02	1	2
2019-01-03	1	3
2019-01-04	1	4
2019-01-05	1	5
2019-01-06	1	6
2019-01-07	1	7
2019-01-08	1	8
2019-01-02	2	1
2019-01-03	3	1
2019-01-01	4	1
2019-01-04	4	2
2019-01-05	5	1
2019-01-06	6	1
2019-01-07	7	1
2019-01-08	8	1
2019-01-01	9	1
2019-01-02	9	2
2019-01-03	9	3
2019-01-04	9	4
2019-01-06	9	5
2019-01-09	9	6
2019-01-10	9	7
2019-01-13	9	8

3日期减去计数值得到结果(简化，没用datediff考虑跨年跨月)

```
C
as( select *,date_sub(
日期, INTERVAL cum DAY)
as 结果 from (B)) #允许列名为UTF字符
```

日期	id	cum	结果
2019-01-01	1	1	20190100
2019-01-02	1	2	20190100
2019-01-03	1	3	20190100
2019-01-04	1	4	20190100
2019-01-05	1	5	20190100
2019-01-06	1	6	20190100
2019-01-07	1	7	20190100
2019-01-08	1	8	20190100
2019-01-02	2	1	20190101
2019-01-03	3	1	20190102
2019-01-01	4	1	20190100
2019-01-04	4	2	20190102
2019-01-05	5	1	20190104
2019-01-06	6	1	20190105
2019-01-07	7	1	20190106
2019-01-08	8	1	20190107
2019-01-01	9	1	20190100
2019-01-02	9	2	20190100
2019-01-03	9	3	20190100
2019-01-04	9	4	20190100
2019-01-06	9	5	20190101
2019-01-09	9	6	20190103
2019-01-10	9	7	20190103
2019-01-13	9	8	20190105

4大于等于7的即为连续登陆7天的用户

```
select id,count(*) from (C) as tmpc
GROUP BY id,结果 having count(*)>=7;
```

id	count(*)
1	8

按成绩排名发放福利

例题：按各科(c_id)成绩进行排序，并显示排名， Score 相同时需保留名次空缺（如果两个分数相同，则排名（rank）相同,下一个名次要考虑同分。例如名次1、1、1、4、4、6**）**

```
select *
from sc a
left join sc b #a表最高分
on a.cid = b.cid
and a.score < b.score #
order by a.sid,a.cid
```

SId	CId	score	SId1	CId1	score1
01	01	80.0	(Null)	(Null)	(Null)
01	02	90.0	(Null)	(Null)	(Null)
01	03	99.0	(Null)	(Null)	(Null)
02	01	70.0	03	01	80.0
02	01	70.0	05	01	76.0
02	01	70.0	01	01	80.0
02	02	60.0	01	02	90.0
02	02	60.0	05	02	87.0
02	02	60.0	07	02	89.0
02	02	60.0	03	02	80.0
02	03	80.0	07	03	98.0

01学生的3门课均为最高分

排名为4，group为前三字段

count(b.score) + 1

https://blog.csdn.net/liyang_nash

```
select a.*,count(b.score)+1 as rank from sc a left join sc b on a.score<b.score and a.cid = b.cid group by a.cid, a.sid,a.score
/*count该课程各学生的分数排名*/
order by a.cid, rank;
```

SId	CId	score	rank
01	01	80	1
03	01	80	1
05	01	76	3
02	01	70	4
04	01	50	5
06	01	31	6
01	02	90	1
07	02	89	2
05	02	87	3
03	02	80	4
02	02	60	5
04	02	30	6

n日内留存率

计算某日的客户在某个时间段内再次出现的比率，称为用户n日内留存率，要求统计用户次日内、3日内、7日内,...留存/留存率

订单总表包括下单时间、客户编号两列

CONCAT(s1,s2...sn)	字符串 s1,s2 等多个字符串合并为一个字符串
ROUND(x,y)	按Y位小数，对X四舍五入
DATE()	从日期或日期时间表达式中提取日期值
CAST(x AS type)	转换数据类型CAST(a.下单时间 AS DATE);

期望结果：

信息 | Result 1 | Result 2 | 概况 | 状态

日期	日活跃用户数	次日内留存用户	三日内留存用户	七日内留存用户	次日内留存率	三日内留存率	七日内留存率
2020-09-01	30	0	1	4	0.00%	3.33%	13.33%
2020-09-02	448	18	34	82	4.02%	7.59%	18.30%
2020-09-03	573	4	15	81	0.70%	2.62%	14.14%
2020-09-04	449	13	21	89	2.90%	4.68%	19.82%
2020-09-05	650	0	53	136	0.00%	8.15%	20.92%

```
select *,
 concat(round(100*次日内留存用户/日活跃用户数,2),'%')  次日内留存率,
 concat(round(100*三日内留存用户/日活跃用户数,2),'%')  三日内留存率,
 concat(round(100*七日内留存用户/日活跃用户数,2),'%')  七日内留存率
from (
 select   /*次日的、留存的、用户数量*/
  date(a.下单时间) as 日期,
  count(distinct a.客户编号) as 日活跃用户数,
  count(distinct b.客户编号) as 次日内留存用户, #数量
  count(distinct c.客户编号) as 三日内留存用户,
  count(distinct d.客户编号) as 七日内留存用户
 from 订单总表 a
  left join 订单总表 b
   on a.客户编号 = b.客户编号   #留存
   and datediff(b.下单时间, a.下单时间) =1 #次日
  left join 订单总表 c
   on a.客户编号 = c.客户编号
   and datediff(b.下单时间, a.下单时间)  BETWEEN 1 AND 3 #3日内留存
  left join 订单总表 d
   on a.客户编号 = d.客户编号
   and datediff(b.下单时间, a.下单时间)  BETWEEN 1 AND 7 #7日内留存
 where date(a.下单时间) between "2020/09/01" and "2020/09/05"
 group by date(a.下单时间)
) p;
```

SELECT语句复习

SELECT [ALL|DISTINCT] #找什么

<目标列表达式> [别名] [,<目标列表达式> [别名]] …

FROM <表名或视图名> [别名] #从哪儿找

[,<表名或视图名> [别名]] …

|(<SELECT语句>)[AS]<别名> # 临时表也可以做条件

[WHERE <条件表达式>] #列的条件

[GROUP BY <列名1>[HAVING<条件表达式>]]

[ORDER BY <列名2> [ASC|DESC]];

LIMIT OFFSET

1. 目标列表达式的可选格式

- 目标列表达式格式

  （1） *

  （2） <表名>.*

  （3） COUNT([DISTINCT|ALL]*)

  （4） [<表名>.]<属性列名表达式>[,<表名>.]<属性列名表达式>]…

其中<属性列名表达式>可以是由属性列、作用于属性列的聚集函数和常量的任意算术运算（+，-，*，/）组成的运算公式

2. 聚集函数的一般格式

{ COUNT
SUM
AVG
MAX
MIN } ([DISTINCT|ALL] <列名>)

3. WHERE子句的条件表达式的可选格式

(1)

<属性列名> θ { <属性列名> | <常量> | [ANY|ALL] （SELECT语句） }

(2)

<属性列名> [NOT] BETWEEN { <属性列名> | <常量> | (SELECT语句) } AND { <属性列名> | <常量> | (SELECT语句) }

WHERE子句的条件表达式格式（续）

（3）

<属性列名> [NOT] IN { (<值1>[, <值2>] …) | (SELECT语句) }

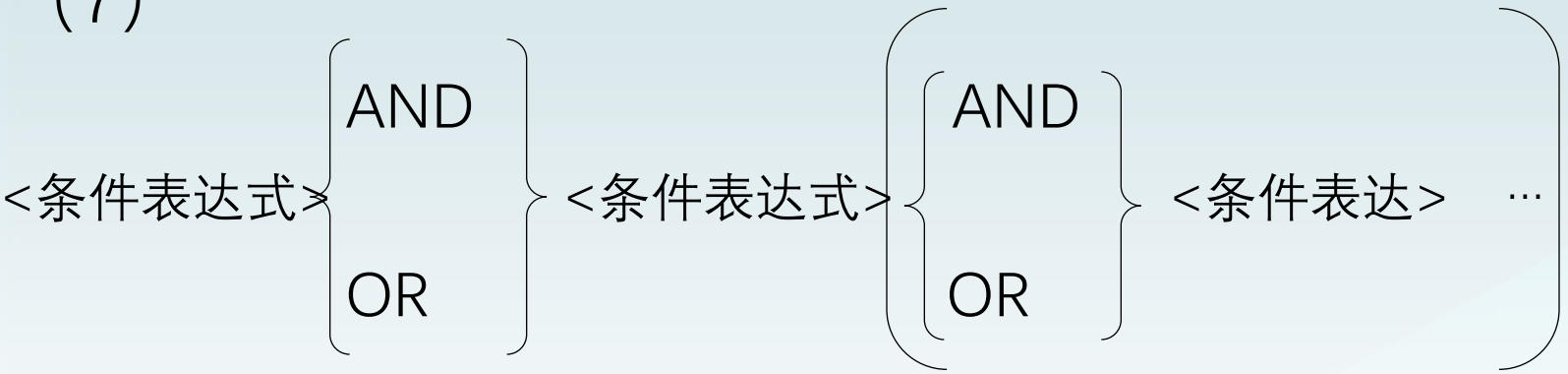
（4） <属性列名> [NOT] LIKE <匹配串>

（5） <属性列名> IS [NOT] NULL

（6） [NOT] EXISTS （SELECT语句）

WHERE子句的条件表达式格式（续）

(7)



多表查询常用方法对比-考试不限

	多表连接	Join	派生表	嵌套查询(相关子查询时用)	并(列相同的场景)
可读性	好	尚可	最适合长代码	多层时不好	好
调试查错	尚可	较方便	方便	较方便(除相关子查询)	较方便
商用性能	商用少用	**商用常用**	性能没问题时,长代码常用	商用常用	能用时用

Mysql编译器常见独特问题

1. 常用sql语句为一个语句，多个错误一般仅报错一处，而且未必是错误所在。
2. 使用group by时，select出现多于group by的非聚集函数列，由于已分组折叠，因此数据具有误导性，但编译器不告警。类似还有max/min聚集函数，并不能直接查询对应的行，但编译器不告警。多表连接缺少连接条件时，出现笛卡尔积也不告警，需掌握语法。
3. 不支持输出打印信息，不支持查看内存。
4. 相关子查询的隐式循环不受控，难调试，需语法正确。
5. 因此需要掌握拆语句调试(join、不相关子查询和派生表容易拆分语句)的技巧，因为有的错误，编译器不报错不告警，需掌握常用语法。

select易错点

- 派生表、不相关子查询，可以互相替代;
- 多表连接、Join、派生表、嵌套(子)查询均可替代，交并差也可用于某些场景，不推荐临时表或视图方式(用派生表)，商用时根据调试查错和性能来选择;
- exists需要外表代入(相关子查询，循环)，而IN一般是不相关子查询(先进行内表查询，也可以是相关子查询，但此时性能一般低于exists);
- 实现某个集合包含另外一个集合(sql除法)，一般使用双not exists;

Select重难点常见场景，不再举例

- group by/聚集函数：常用于需要分组的场景，然后聚集函数求值;
- 自身连接(局限性大，灵活需要用)：同表的数据需要连接才能间接匹配(间接先修课);
- 相关子查询(隐含两层循环)：外层代入内层循环;
- 双not exists：求包含某个集合的元素的集合。
- 派生表：构造一个结果集，也可替代某些相关子查询。

课后习题

- 第四章课后习题：1，2，3
  - 写SQL作答
- 提交链接：微信群课后分享